TP 2 — Création et Gestion d'API REST Avancées en Java avec Spring Boot

Ralph Bou Nader

Master 2 — Ingénierie Informatique

Objectifs

Ce TP avancé renforce vos compétences de création d'API REST en Java avec Spring Boot, en couvrant : la persistance de données, la validation, la gestion des erreurs, l'authentification, mais aussi des pratiques modernes indispensables en production — documentation automatique, pagination, limitation de débit, conteneurisation et tests. Vous utiliserez Postman pour tester chaque point de terminaison, comme au TP1.

Prérequis

- Java JDK 17 ou plus récent (Spring Boot 3.x exige au minimum Java 17).
- Maven (ou Gradle).
- Spring Boot 3.x, généré via Spring Initializr.
- Postman.
- MySQL ou PostgreSQL (ou toute autre base de données relationnelle).
- Docker Desktop (pour la Section 6, nouvelle).

Remarque sur les espaces de nommage : Spring Boot 3.x est basé sur Jakarta EE — les imports utilisent désormais `jakarta.persistence.*` et `jakarta.validation.*`, et non plus `javax.*` comme dans les versions antérieures. Veillez à utiliser les bonnes dépendances dans votre `pom.xml`.

Exercice 1 — API de Calculatrice Avancée

Objectif : créer une API REST gérant des opérations mathématiques avec validation des données et historique persistant.

Mise en place

1. Créez un projet Spring Boot avec les dépendances `Spring Web`, `Spring Data JPA`, `Validation`, et le pilote de votre base de données.
2. Créez une entité `Operation` avec les champs : `id`, `operateur` (addition, soustraction, multiplication, division), `a`, `b`, `resultat`, `date`.
3. Implémentez un `OperationRepository` (interface `JpaRepository`) pour persister les opérations.
4. Créez un contrôleur `CalculatorController` avec :
   - `POST /api/calculator` : accepte `a`, `b`, `operation` en JSON, valide les entrées, calcule et renvoie le résultat, et sauvegarde l'opération.
   - `GET /api/calculator/history` : renvoie l'historique complet des opérations.

Validation et gestion des erreurs

- Ajoutez des contraintes de validation (`@NotNull`, `@DecimalMin`, `@DecimalMax`) sur les champs `a` et `b` du DTO de requête.
- Interdisez explicitement la division par zéro : si `operation = "division"` et `b = 0`, renvoyez une erreur `400 Bad Request` avec un message explicite plutôt qu'une exception non gérée.
- Implémentez un gestionnaire d'exceptions global avec `@RestControllerAdvice` et `@ExceptionHandler`, pour que **toutes** les erreurs de l'API (validation, division par zéro, ressource introuvable) renvoient un corps JSON structuré et cohérent, par exemple :

```
{
  "timestamp": "2026-09-20T10:15:30",
  "status": 400,
  "error": "Bad Request",
  "message": "La division par zéro n'est pas autorisée",
  "path": "/api/calculator"
}
```

Cette approche du gestionnaire d'erreurs centralisé est absente de la version précédente de ce TP — c'est aujourd'hui une pratique standard en Spring Boot, à ne plus traiter au cas par cas dans chaque contrôleur.

CRUD complet

- `PUT /api/calculator/{id}` : met à jour une opération existante.
- `DELETE /api/calculator/{id}` : supprime une opération de l'historique.

Pagination de l'historique *(nouveau)*

L'endpoint `GET /api/calculator/history` peut devenir coûteux si l'historique contient des milliers d'entrées — un problème directement lié à l'*over-fetching* vu en cours à propos de GraphQL, mais qui existe aussi côté REST. Modifiez cet endpoint pour accepter des paramètres de pagination Spring Data (`Pageable`), utilisables ainsi :

```
GET /api/calculator/history?page=0&size=10&sort=date,desc
```

Vérifiez avec Postman que la réponse contient bien les métadonnées de pagination (nombre total d'éléments, nombre de pages…) en plus du contenu.

Exercice 2 — API de Conversion de Devises avec Taux Dynamiques

Objectif : créer une API REST convertissant des devises via des taux de change récupérés depuis une source externe, en gérant proprement les défaillances de cette dépendance externe.

Mise en place

1. Créez un contrôleur `CurrencyConverterController`.
2. Implémentez `GET /api/currency/convert?amount=...&from=...&to=...`, qui appelle un service externe de taux de change (par exemple une API publique gratuite comme exchangerate-api.com ou frankfurter.app) via `RestTemplate` ou, de préférence, le client HTTP réactif moderne `WebClient`.
3. Créez un service dédié pour gérer la logique de conversion et persister chaque transaction en base.
4. Mettez en cache les taux récupérés (annotation `@Cacheable` de Spring) afin de réduire le nombre d'appels vers l'API externe pour des devises fréquemment demandées.

CRUD et robustesse

- `POST /api/currency/convert` : permet de spécifier un taux de conversion manuel pour des devises non couvertes par l'API externe.

- `PUT /api/currency/update-rate` : met à jour un taux stocké localement.
- `DELETE /api/currency/delete-transaction/{id}` : supprime une transaction.
- **Transactions et rollback** : utilisez `@Transactional` pour garantir qu'une conversion n'est enregistrée en base que si l'appel externe et le calcul ont réussi.
- **Gestion des erreurs et retry** : si l'API externe est indisponible ou répond lentement, implémentez une logique de nouvelle tentative (`@Retryable` de Spring Retry, ou une boucle manuelle avec backoff) avant d'abandonner et de renvoyer une erreur `503 Service Unavailable` explicite au client.

Limitation de débit *(nouveau, en lien avec le cours de sécurité)*

Pour protéger votre API et l'API externe qu'elle consomme d'un usage abusif, ajoutez une limitation de débit simple sur l'endpoint `GET /api/currency/convert` : par exemple 20 requêtes par minute et par adresse IP, à l'aide d'un filtre Spring personnalisé ou de la bibliothèque `bucket4j`. Testez ce comportement avec Postman en envoyant rapidement plusieurs requêtes successives (utilisez le **Collection Runner** de Postman pour automatiser l'envoi répété) et vérifiez qu'un code `429 Too Many Requests` est bien renvoyé au-delà du seuil.

Exercice 3 — API de Gestion du BMI avec Authentification et Statistiques

Objectif : calculer et stocker le BMI (indice de masse corporelle) de différents utilisateurs, en sécurisant l'accès avec une authentification par jeton JWT.

Mise en place

1. Créez une entité `User` (`id`, `username`, `password` haché, `email`).
2. Implémentez un service d'authentification délivrant un JWT (bibliothèque `jjwt` ou équivalent) au format vu en cours : `Header.Payload.Signature`.
3. Créez une entité `BMIRecord` (`id`, `userId`, `weight`, `height`, `bmi`).
4. Implémentez `BMICalculatorController` avec :
   - `POST /api/bmi/calculate` : calcule et enregistre le BMI de l'utilisateur authentifié.
   - `GET /api/bmi/history` : renvoie l'historique des calculs de l'utilisateur authentifié.

Fonctionnalités avancées

- `PUT /api/bmi/update/{id}` : met à jour poids/taille d'un enregistrement et recalcule le BMI.
- `DELETE /api/bmi/delete/{id}` : supprime un enregistrement, après avoir vérifié que l'utilisateur authentifié en est bien le propriétaire (rappel du cours — autorisation, pas seulement authentification).
- `GET /api/bmi/statistics` : statistiques globales (BMI moyen, répartition par catégorie).
- **Refresh tokens** : implémentez un mécanisme de renouvellement du jeton d'accès sans réauthentification complète de l'utilisateur, comme vu en cours.

Sécuriser tous les points de terminaison

Assurez-vous, avec Spring Security, que **tous** les endpoints de cet exercice exigent un JWT valide, à l'exception de l'inscription et de la connexion. Testez avec Postman : une requête sans en-tête `Authorization` doit renvoyer `401 Unauthorized` ; une requête tentant de supprimer l'enregistrement d'un autre utilisateur doit renvoyer `403 Forbidden`.

Exercice 4 — Documentation, Conteneurisation et Tests *(exercice entièrement nouveau)*

Objectif : appliquer les pratiques attendues d'une API prête pour la production, au-delà du seul code fonctionnel.

Documentation automatique avec OpenAPI/Swagger

1. Ajoutez la dépendance `springdoc-openapi-starter-webmvc-ui` à votre projet.
2. Lancez l'application et ouvrez `http://localhost:8080/swagger-ui.html` : vous devez y voir tous les endpoints des trois exercices précédents, générés automatiquement à partir de votre code.
3. Enrichissez au moins trois endpoints avec les annotations `@Operation` et `@ApiResponse` pour préciser leur description et leurs réponses possibles.

Conteneuriser l'application avec Docker

1. Écrivez un `Dockerfile` multi-étapes (build Maven puis image d'exécution légère basée sur un JRE) pour votre application.
2. Construisez l'image (`docker build`) et lancez un conteneur (`docker run`), en vérifiant que l'API reste accessible et fonctionnelle depuis Postman.
3. *(Facultatif, niveau expert)* Écrivez un `docker-compose.yml` démarrant à la fois l'application et sa base de données en une seule commande.

Tests automatisés

1. Écrivez au moins trois tests avec JUnit 5 et `MockMvc` couvrant : un cas de succès (ex. `POST /api/calculator` avec des données valides), un cas d'erreur de validation (ex. division par zéro), et un cas d'accès non autorisé (ex. `GET /api/bmi/history` sans jeton).
2. Exécutez la suite de tests et incluez une capture du résultat dans votre rapport.

Rapport et Livrables

Soumettez :

1. Le code source complet du projet (dépôt Git ou archive).
2. La structure de la base de données (script SQL ou capture du schéma généré).
3. Des captures d'écran Postman pour **chaque** endpoint des quatre exercices (requête + réponse), y compris les cas d'erreur (400, 401, 403, 404, 429).
4. Le fichier `Dockerfile` (et `docker-compose.yml` si réalisé).
5. Le résultat de l'exécution de vos tests JUnit.
6. Un rapport décrivant les défis rencontrés, les choix d'implémentation, et une réflexion sur l'apport de la documentation OpenAPI et de la conteneurisation pour une API destinée à la production.

Barème indicatif

Critère	Points
Exercice 1 — Calculatrice (CRUD, validation, pagination)	4
Exercice 2 — Devises (appel externe, transactions/rollback, rate limiting)	5
Exercice 3 — BMI (JWT, autorisation, statistiques)	5
Exercice 4 — Documentation, Docker, tests	4
Qualité du rapport et du code	2
Total	**20**